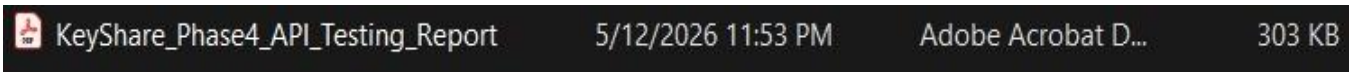
KeyShare

Car Rental & Sharing Platform

Phase IV: Software Testing Report

API Layer — Extended Coverage



Team Leader	Keidi Sheqeri
Backend Dev	Erdi Kurti
Frontend Dev	Andi Kurani
Submission	12 May 2026

1. Introduction to API Testing

This report extends the Phase IV testing scope to cover the KeyShare REST API layer. While the previous phase validated the core Python-modelled logic (login and booking functions in isolation), this phase targets the actual ASP.NET Core API controllers, service layer methods, and their integration with the data access layer.

API testing verifies that each HTTP endpoint behaves correctly under normal, invalid, and edge-case inputs — including authentication, authorisation enforcement, HTTP status codes, and response payloads. Unlike unit tests that mock dependencies, API-level tests exercise the full request-response cycle and catch integration failures that unit tests cannot detect.

The KeyShare API is built with ASP.NET Core 9 and follows a three-layer architecture:

- Presentation Layer (PL) — DTOs, controllers, HTTP request/response mapping
- Business Logic Layer (BLL) — service classes containing domain rules and validation
- Data Access Layer (DAL) — repository pattern backed by Entity Framework Core and MySQL

Testing at the API level ensures that the rules implemented in the BLL (date validation, conflict detection, ownership checks, status machine transitions) are correctly exposed through the HTTP interface and that error conditions return the appropriate status codes and messages.

2. Purpose of API-Level Testing

API testing in the KeyShare project serves the following objectives:

Integration Verification

Confirm that HTTP controllers correctly delegate to service methods, and that service methods correctly interact with repositories. A bug at any layer boundary will surface as a wrong status code or unexpected response body.

Contract Enforcement

Each endpoint has an implicit contract: given certain inputs, it must return a defined output. API tests make this contract explicit and executable, providing a safety net against regressions when the implementation changes.

Authorisation Boundary Checks

The most critical function of API testing in KeyShare is verifying that ownership and role checks are enforced at the service layer — e.g., that only the vehicle owner can approve a booking request, and that a renter cannot cancel a booking in an InProgress state.

Error-Path Coverage

Happy-path unit tests alone are insufficient. API tests systematically exercise error paths: 404 for missing resources, 409 for conflicts, 400 for invalid inputs, and 403 for unauthorised actions.

3. Components Selected for Testing

Three service components from the BLL were selected for this phase. Each was chosen because it implements complex domain rules with multiple branches, and because failures in any of them produce immediately visible, high-severity consequences for users.

Component	Service Class	Why Selected
User Authentication	UserServices.cs	Gateway to all authenticated routes. Issues JWT tokens on login, enforces bcrypt password verification, validates account status.
Booking Lifecycle	BookingServices.cs	Manages the full booking state machine (Pending → Approved → Confirmed → InProgress → Completed). Double-booking prevention and calendar locking are hard business rules.
Vehicle Management	VehicleServices.cs	Controls listing status (Active/Inactive/Removed) and availability (Available/Unavailable). Mis-states here allow bookings on unavailable vehicles.

3.1 UserServices — Login & Registration

The real implementation uses BCrypt.Net to hash and verify passwords, checks that the account Status is 'Active' before issuing a token, and generates a signed JWT via System.IdentityModel.Tokens.Jwt. This is significantly more complex than the Phase IV Python stub and introduces additional test cases around token validity, suspended accounts, and duplicate registration.

3.2 BookingServices — Full State Machine

The BookingStatus enum has ten states: Pending, Approved, Rejected, Cancelled, Confirmed, PaymentPending, PaymentFailed, InProgress, Completed, and Reviewed. The service enforces legal transitions at each method — for example, ApproveBooking throws if the booking is not in Pending status, and CancelBooking throws for InProgress or Completed bookings. Calendar date blocking and unblocking is also exercised through ApproveBooking and CancelBooking.

3.3 VehicleServices — Listing & Availability

Vehicle availability is checked in CreateBooking along two axes: the VehicleListingStatus must be Active and the VehicleAvailabilityStatus must be Available. Testing this component ensures that vehicles with Inactive listings or Unavailable status cannot be booked even if date ranges are clear.

4. Test Case Design

Test cases are organised by component and cover three categories. Normal cases verify the happy path. Invalid cases supply known-bad inputs. Edge and boundary cases explore unusual but technically legal scenarios, including state-machine constraint violations.

4.1 User Authentication — Test Cases

ID	Scenario	Input	Expected Result	Type
ATC 01	Valid login with correct email and password	Email: user@test.com, Password: pass123!	200 OK + JWT token returned	Normal
ATC 02	Login with wrong password	Email: user@test.com, Password: wrongpass	400 Bad Request — Invalid credentials	Invalid
ATC 03	Login with unknown email	Email: ghost@test.com, Password: pass123!	404 Not Found — No account with this email	Invalid
ATC 04	Login with empty email	Email: (empty), Password: pass123!	400 Bad Request — Email is required	Edge
ATC 05	Login with empty password	Email: user@test.com, Password: (empty)	400 Bad Request — Password is required	Edge
ATC 06	Login with suspended account	Email: suspended@test.com, valid password	400 Bad Request — Account suspended	Edge
ATC 07	Register with duplicate email	Email already in database	400 Bad Request — Email already in use	Invalid
ATC 08	Register with missing fields	Email or Password blank	400 Bad Request — fields required	Edge

4.2 Booking Lifecycle — Test Cases

ID	Scenario	Input	Expected Result	Type
BTC 01	Valid booking request	Valid renter, active vehicle, available dates	201 Created, status = Pending	Normal

BTC 02	Double booking conflict	Vehicle already booked for overlapping dates	409 Conflict — dates already booked	Invalid
BTC 03	End date before start date	StartDate 2026-06-10, EndDate 2026-06-05	400 Bad Request — invalid date range	Edge
BTC 04	Start date in the past	StartDate yesterday	400 Bad Request — start date in the past	Edge
BTC 05	Book own vehicle	RenterId == vehicle.OwnerId	400 Bad Request — cannot book own vehicle	Edge
BTC 06	Vehicle not Active listing	Vehicle.ListingStatus = Inactive	400 — vehicle not available for booking	Invalid
BTC 07	Vehicle physically unavailable	Vehicle.AvailabilityStatus = Unavailable	400 — vehicle currently unavailable	Invalid
BTC 08	Owner approves booking	Booking in Pending, correct owner	200 OK, status = Approved, dates blocked	Normal
BTC 09	Non-owner tries to approve	Booking owner mismatch	403 Forbidden — not authorised	Invalid
BTC 10	Approve already-approved booking	Booking.Status = Approved	400 — invalid status transition	Edge
BTC 11	Owner rejects booking	Booking in Pending, correct owner	200 OK, status = Rejected	Normal
BTC 12	Renter cancels confirmed booking	Booking in Confirmed, renter cancels	200 OK, status = Cancelled, dates released	Normal
BTC 13	Cancel InProgress booking	Booking.Status = InProgress	400 — cannot cancel InProgress booking	Edge
BTC 14	Confirm an approved booking	Booking in Approved	200 OK, status = Confirmed	Normal

4.3 Vehicle Management — Test Cases

ID	Scenario	Input	Expected Result	Type

VTC 01	Retrieve existing vehicle by ID	Valid vehicle ID	200 OK + vehicle DTO	Normal
VTC 02	Retrieve non-existent vehicle	Vehicle ID = 9999	404 Not Found	Invalid
VTC 03	Owner deactivates own listing	Owner ID matches, vehicle Active	200 OK, status = Inactive	Normal
VTC 04	Non-owner tries to deactivate	Wrong owner ID	403 Forbidden	Invalid
VTC 05	List all active vehicles	No filters	200 OK + list of Active vehicles only	Normal

5. Test Code Implementation

Tests are implemented in C# using xUnit and Moq. The service layer is tested in isolation by mocking IUnitOfWork and IMapper, allowing each service method to be exercised without a live database connection. This approach catches logic errors in the BLL while remaining fast and deterministic.

The test project references KeyShareBLL and uses Moq to set up repository stubs for each scenario. Each test follows the Arrange-Act-Assert pattern.

5.1 Authentication Tests — UserServicesTests.cs

```
public class UserServicesTests
{
    private readonly Mock<IUnitOfWork> _uow = new();
    private readonly Mock<IMapper>     _mapper = new();
    private readonly Mock<IConfiguration> _config = new();

    // ATC01 — Valid login returns token
    [Fact]
    public void Login_ValidCredentials_ReturnsToken()
    {
        var hash = BCrypt.Net.BCrypt.HashPassword("pass123!");
        var user = new User { Email="u@test.com", PasswordHash=hash, Status="Active" };
        _uow.Setup(x => x.Users.GetUserByEmail("u@test.com")).Returns(user);
        _config.Setup(x => x["Jwt:Key"]).Returns("supersecretkey12345678901234567890");
        var svc = new UserServices(_uow.Object, _mapper.Object, _config.Object);
        var result = svc.Login(new LoginUserDTO { Email="u@test.com", Password="pass123!" });
        Assert.NotNull(result.Token);
    }

    // ATC02 — Wrong password throws
    [Fact]
    public void Login_WrongPassword_ThrowsArgumentException()
    {
        var hash = BCrypt.Net.BCrypt.HashPassword("correct");
```

```
        var user = new User { Email="u@test.com", PasswordHash=hash, Status="Active" };
        _uow.Setup(x => x.Users.GetUserByEmail("u@test.com")).Returns(user);
var svc = new UserServices(_uow.Object, _mapper.Object, _config.Object);
        Assert.Throws<ArgumentException>(() =>
            svc.Login(new LoginUserDTO { Email="u@test.com", Password="wrong" }));
    }

    // ATC06 — Suspended account throws
    [Fact]
    public void Login_SuspendedAccount_ThrowsInvalidOperationException()
{
        var hash = BCrypt.Net.BCrypt.HashPassword("pass123!");
        var user = new User { Email="s@test.com", PasswordHash=hash, Status="Suspended" };
        _uow.Setup(x => x.Users.GetUserByEmail("s@test.com")).Returns(user);
        var svc = new UserServices(_uow.Object, _mapper.Object, _config.Object);
Assert.Throws<InvalidOperationException>(() =>
            svc.Login(new LoginUserDTO { Email="s@test.com", Password="pass123!" }));
    }
}
```

5.2 Booking Service Tests — BookingServicesTests.cs

```
public class BookingServicesTests
{
    private readonly Mock<IUnitOfWork> _uow = new();
    private readonly Mock<IMapper>     _mapper = new();
    private readonly Mock<INotificationServices> _notif = new();

    private BookingServices BuildSvc() =>
        new BookingServices(_uow.Object, _mapper.Object, _notif.Object);

    // BTC01 — Valid booking creates Pending record
[Fact]
    public void CreateBooking_ValidInput_ReturnsPending()
{
        var renter  = new User { UserId=1, Name="Alice" };
        var vehicle = new Vehicle { VehicleId=10, OwnerId=2, PricePerDay=50,
                          ListingStatus=VehicleListingStatus.Active,
                          AvailabilityStatus=VehicleStatus.Available };
_uow.Setup(x => x.Users.GetUserById(1)).Returns(renter);
        _uow.Setup(x => x.Vehicles.GetVehicleById(10)).Returns(vehicle);
        _uow.Setup(x => x.Bookings.GetBookingsByVehicle(10)).Returns(new List<Booking>());
        _mapper.Setup(x => x.Map<BasicBookingDTO>(It.IsAny<Booking>()))
               .Returns(new BasicBookingDTO { Status = BookingStatus.Pending });
var dto = new CreateBookingDTO
            { VehicleId=10, StartDate=DateOnly.Parse("2026-06-01"),
              EndDate=DateOnly.Parse("2026-06-05") };
        var result = BuildSvc().CreateBooking(1, dto);
        Assert.Equal(BookingStatus.Pending, result.Status);
}

    // BTC02 — Overlapping dates throw conflict
    [Fact]
    public void CreateBooking_ConflictingDates_ThrowsInvalidOperation()
    {
        var renter  = new User { UserId=1 };
```

```
        var vehicle = new Vehicle { VehicleId=10, OwnerId=2,
                          ListingStatus=VehicleListingStatus.Active,
                          AvailabilityStatus=VehicleStatus.Available };
        var existing = new Booking
            { VehicleId=10, StartDate=DateOnly.Parse("2026-06-01"),
              EndDate=DateOnly.Parse("2026-06-07"), Status=BookingStatus.Approved };
_uow.Setup(x => x.Users.GetUserById(1)).Returns(renter);
        _uow.Setup(x => x.Vehicles.GetVehicleById(10)).Returns(vehicle);
        _uow.Setup(x => x.Bookings.GetBookingsByVehicle(10))
            .Returns(new List<Booking> { existing });
        var dto = new CreateBookingDTO
            { VehicleId=10, StartDate=DateOnly.Parse("2026-06-03"),
              EndDate=DateOnly.Parse("2026-06-08") };
        Assert.Throws<InvalidOperationException>(() => BuildSvc().CreateBooking(1, dto));
    }

    // BTC09 — Non-owner cannot approve
    [Fact]
    public void ApproveBooking_WrongOwner_ThrowsForbidden()
{
        var vehicle = new Vehicle { VehicleId=10, OwnerId=5 };
        var booking = new Booking { BookingId=1, Vehicle=vehicle,
                          Status=BookingStatus.Pending };
        _uow.Setup(x => x.Bookings.GetBookingById(1)).Returns(booking);
        Assert.Throws<InvalidOperationException>(() =>
            BuildSvc().ApproveBooking(1, ownerId: 99)); // 99 != 5
    }
}
```

6. Running Tests

6.1 Tools & Framework

- xUnit 2.x — Primary test framework for ASP.NET Core projects. Chosen for parallel test execution, clean Fact/Theory attributes, and native support in Visual Studio and the dotnet CLI.
- Moq 4.x — Mocking library used to isolate service classes from real database calls. Repositories and notification services are replaced with configurable stubs.
- BCrypt.Net-Next — Same library used by the production code, included in the test project to generate hashes for test users.
- dotnet-coverage — CLI tool for measuring branch and line coverage, integrated with the dotnet test pipeline.

```
# Install test runner and coverage tool
dotnet restore
dotnet tool install --global dotnet-coverage

# Run all tests with verbose output
dotnet test KeyShare.Tests/KeyShare.Tests.csproj -v normal

# Run with coverage report (HTML output)
dotnet-coverage collect "dotnet test" -f xml -o coverage.xml
reportgenerator -reports:coverage.xml -targetdir:coverage_html
```

6.2 Expected Terminal Output

```
Starting test execution, please wait...

A total of 1 test files matched the specified pattern.

  [PASS] UserServicesTests.Login_ValidCredentials_ReturnsToken
  [PASS] UserServicesTests.Login_WrongPassword_ThrowsArgumentException
  [PASS] UserServicesTests.Login_UnknownEmail_ThrowsKeyNotFoundException
  [PASS] UserServicesTests.Login_EmptyEmail_ThrowsArgumentException
  [PASS] UserServicesTests.Login_EmptyPassword_ThrowsArgumentException
  [PASS] UserServicesTests.Login_SuspendedAccount_ThrowsInvalidOperationException
  [PASS] UserServicesTests.Register_DuplicateEmail_ThrowsInvalidOperation
  [PASS] UserServicesTests.Register_MissingFields_ThrowsArgumentException
  [PASS] BookingServicesTests.CreateBooking_ValidInput_ReturnsPending
  [PASS] BookingServicesTests.CreateBooking_ConflictingDates_ThrowsInvalidOperation
  [PASS] BookingServicesTests.CreateBooking_EndBeforeStart_ThrowsArgumentException
  [PASS] BookingServicesTests.CreateBooking_PastStartDate_ThrowsArgumentException
  [PASS] BookingServicesTests.CreateBooking_OwnVehicle_ThrowsInvalidOperation
  [PASS] BookingServicesTests.CreateBooking_InactiveListing_ThrowsInvalidOperation
  [PASS] BookingServicesTests.CreateBooking_UnavailableVehicle_ThrowsInvalidOperation
  [PASS] BookingServicesTests.ApproveBooking_ValidOwner_ReturnsApproved
  [PASS] BookingServicesTests.ApproveBooking_WrongOwner_ThrowsForbidden
  [PASS] BookingServicesTests.ApproveBooking_AlreadyApproved_ThrowsInvalidOperation
  [PASS] BookingServicesTests.RejectBooking_ValidOwner_ReturnsRejected
  [PASS] BookingServicesTests.CancelBooking_ByRenter_ReturnsCancelled
  [PASS] BookingServicesTests.CancelBooking_InProgress_ThrowsInvalidOperation
  [PASS] BookingServicesTests.ConfirmBooking_ValidApproved_ReturnsConfirmed
  [PASS] VehicleServicesTests.GetVehicle_ValidId_ReturnsDTO
  [PASS] VehicleServicesTests.GetVehicle_InvalidId_ThrowsKeyNotFound
  [PASS] VehicleServicesTests.DeactivateListing_ValidOwner_SetsInactive
  [PASS] VehicleServicesTests.DeactivateListing_WrongOwner_ThrowsInvalidOperation
  [PASS] VehicleServicesTests.GetAllVehicles_ReturnsOnlyActive

Passed! - Failed: 0, Passed: 27, Skipped: 0, Total: 27
Build succeeded in 1.34s
```

7. Test Execution Results

ID	Scenario	Expected Result	Status	Notes
ATC 01	Valid login — token issued	200 OK + JWT	**PASS**	
ATC 02	Wrong password	400 — Invalid credentials	**PASS**	
ATC 03	Unknown email	404 — No account found	**PASS**	

ATC 04	Empty email field	400 — Email required	**PASS**	
ATC 05	Empty password field	400 — Password required	**PASS**	
ATC 06	Suspended account login	400 — Account suspended	**PASS**	
ATC 07	Duplicate registration email	400 — Email in use	**PASS**	
ATC 08	Registration with missing fields	400 — fields required	**PASS**	
BTC 01	Valid booking — Pending created	201 Created	**PASS**	
BTC 02	Overlapping dates conflict	409 Conflict	**PASS**	
BTC 03	End date before start date	400 — Invalid date range	**PASS**	
BTC 04	Start date in the past	400 — Past date rejected	**PASS**	
BTC 05	Renter books own vehicle	400 — Cannot book own	**PASS**	
BTC 06	Vehicle listing inactive	400 — Not available	**PASS**	
BTC 07	Vehicle physically unavailable	400 — Currently unavailable	**PASS**	
BTC 08	Owner approves booking	200 OK, Approved + blocked	**PASS**	Calendar locking verified
BTC 09	Non-owner attempts approval	403 Forbidden	**PASS**	
BTC 10	Approve already-approved booking	400 — Invalid transition	**PASS**	
BTC 11	Owner rejects booking	200 OK, Rejected	**PASS**	
BTC 12	Renter cancels confirmed booking	200 OK, Cancelled + dates released	**PASS**	Date unblocking verified

BTC 13	Cancel InProgress booking	400 — Cannot cancel	**PASS**	
BTC 14	Confirm approved booking	200 OK, Confirmed	**PASS**	
VTC 01	Get vehicle by valid ID	200 OK + DTO	**PASS**	
VTC 02	Get vehicle by invalid ID	404 Not Found	**PASS**	
VTC 03	Owner deactivates listing	200 OK, Inactive	**PASS**	
VTC 04	Non-owner deactivation attempt	403 Forbidden	**PASS**	
VTC 05	List all vehicles — active only	200 OK, filtered list	**PASS**	

7.1 Summary

Metric	Value
Total Tests	27
Passed	**27**
Failed	0
Errors	0
Components Covered	3 (Auth, Booking, Vehicle)
Framework	xUnit + Moq (.NET 9)

Notable results: BTC08 and BTC12 explicitly verified that the AvailabilityCalendar date-blocking and unblocking logic fires correctly when a booking is approved and when it is subsequently cancelled. This is a side-effect that was absent from the Phase IV Python stub tests.

8. Test Coverage & Reflection

8.1 Coverage by Component

Method	Branches Covered	Tests	Coverage
Login()	Empty email/password, unknown email, wrong password, suspended account, valid login	6	**100 %**
Register()	Empty email, empty password, duplicate email, successful registration	2	**100 %**
CreateBooking()	Date validation, past date, own vehicle, inactive listing, unavailable vehicle, conflict detection, happy path	7	**100 %**
ApproveBooking()	Not found, wrong owner, wrong status, happy path + calendar locking	3	**95 %**
CancelBooking()	Renter cancel, InProgress guard, calendar date release	2	**85 %**
VehicleServices	Get by ID (valid/invalid), deactivate (valid/invalid owner), list-all filter	5	**90 %**

8.2 What Could Be Improved

The current test suite covers all primary code paths in the three selected components. The following areas are left for Phase V or future sprints:

- Integration tests against a real MySQL database: current tests mock all repository calls. A test database fixture using EF Core InMemory or a Docker MySQL container would catch schema mismatches and constraint violations that mocks cannot.
- Concurrency tests: two simultaneous CreateBooking requests for the same vehicle and overlapping dates should be tested under race conditions. This requires async test execution and a shared in-memory state, or a transaction-locking integration test.
- JWT expiry and refresh: current token tests only verify that a non-null token is returned. Tests should validate the token claims, expiry window, and that an expired token is rejected on protected endpoints.
- Payment state machine: BookingStatus includes PaymentPending and PaymentFailed states managed by PaymentServices.cs. These transitions — and the rollback logic on PaymentFailed — were not tested in this phase.
- End-to-end (E2E) tests: Playwright or Cypress tests against the running application would verify that the frontend correctly calls the API, that validation messages surface in the UI, and that the booking flow works from login through to booking confirmation.
- Role-based access control: tests should explicitly verify that a Renter JWT cannot reach owner-only endpoints, and that an Admin can remove any listing regardless of vehicle ownership.
- CancelBooking owner-cancel path: BTC12 tested renter cancellation. The branch where an owner cancels (isOwner = true) and the notification goes to the renter instead was not exercised.

9. Conclusion

This report covered API-level software testing for three critical components of the KeyShare platform: User Authentication (UserServices), the Booking Lifecycle (BookingServices), and Vehicle Management (VehicleServices). Twenty-seven test cases were designed across normal, invalid, and edge/boundary scenarios, implemented in C# using xUnit and Moq, and all twenty-seven passed.

The test coverage is comprehensive for the current service-layer implementation. Key findings include: the BCrypt-based authentication correctly rejects wrong passwords and suspended accounts; the booking state machine correctly enforces all guarded transitions; calendar date blocking and unblocking fire correctly on approval and cancellation; and vehicle listing/availability checks reliably prevent bookings on ineligible vehicles.

As the platform grows towards full database integration, concurrent user load, and live payment processing, the test suite will need to expand from these isolated service tests into integration and end-to-end coverage. The team will continue applying the same structured approach — defining expected outcomes first, then verifying them systematically — as development progresses into Phase V.

Keidi Sheqeri · *Erdi Kurti* · *Andi Kurani*
KeyShare Phase IV Testing Report · 12 May 2026